

Машинное обучение

Семинар 12

Кластеризация и визуализация

Как вы уже знаете, алгоритмы машинного обучения делятся на два типа — алгоритмы обучения с учителем (*supervised learning*) и алгоритмы обучения без учителя (*unsupervised learning*). К первому типу относятся, например, задачи регрессии и классификации. Чтобы обучить модель предсказывать некоторую скалярную величину или метку класса, требуется датасет, содержащий как признаки, так и целевую переменную. Ко второму типу относятся, в частности, такие задачи, как кластеризация и обучение представлений. В этих случаях у нас отсутствуют правильные ответы, поэтому построение и выбор алгоритмов обучения, а также метрик их оценки, представляет собой нетривиальную задачу.

1 Постановка задачи кластеризации

Задача кластеризации заключается в том, чтобы объединить «похожие» между собой объекты — то есть такие, которые обладают схожими свойствами, — в один кластер, а «непохожие» объекты, наоборот, разделить между разными кластерами. Например, если рассматривать изображения рукописных цифр, то естественно ожидать, что все изображения цифры «3» окажутся в одном кластере, а изображения цифры «8» — в другом. Аналогично, при анализе пользователей онлайн-сервиса можно ожидать, что кластеры будут соответствовать различным группам поведения — например, типичным пользователям, подозрительным аккаунтам или случаям мошенничества.

Однако возникает вопрос: как определить, какие объекты «похожи», а какие — нет? Что вообще означает, что объекты схожи друг с другом? Однозначного ответа на этот вопрос не существует. Именно поэтому разработка методов кластеризации и особенно метрик их качества представляет собой нетривиальную задачу. Тем не менее можно строить алгоритмы, которые выделяют кластеры, исходя из общих интуитивных соображений и предположений о структуре данных.

Пусть у нас имеется набор данных $X = \{x_i\}_{i=1}^n$, где $x_i \in \mathbb{R}^d$ задает признаковое описание i -го объекта $x_i = (x_{i,1}, \dots, x_{i,d})$. На выходе, мы хотим получить алгоритм $a : \mathbb{X} \rightarrow \{1, \dots, K\}$, который бы сопоставил каждому объекту номер его кластера. Число кластеров K обычно выступает в качестве гиперпараметра модели и может быть задано заранее или определяться автоматически в процессе обучения.

В такой постановке задачи сразу возникает несколько вопросов. А именно:

1. Как оценивать качество кластеризации?

2. Как сравнивать два алгоритма кластеризации между собой?
3. Как выбирать гиперпараметр K , если число кластеров заранее не известно?
4. Возможно ли визуализировать разбиение данных на кластеры и как это сделать?

На эти и связанные с ними вопросы мы постараемся ответить в дальнейших разделах.

2 Основные методы кластеризации

В этой секции, приведем несколько основных методов кластеризации.

§2.1 K-Means

Метод **K-Means** разделяет набор данных на K кластеров так, чтобы объекты внутри одного кластера были как можно более похожи друг на друга, а объекты разных кластеров — как можно более различны. В качестве меры сходства используется обычное евклидово расстояние между объектами (их векторами-признаками).

Постановка задачи. Пусть даны объекты $X = \{x_i\}_{i=1}^n$, где $x_i = (x_{i,1}, \dots, x_{i,d})$ обозначает вектор-признаков i -го объекта. Алгоритм **K-Means** стремится найти разбиение X на K кластеров $C_1, \dots, C_K$ и центры кластеров $\mu_1, \dots, \mu_K$ таким образом, чтобы минимизировать внутрикластерную дисперсию (сумму квадратов расстояний до центров кластеров):

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \rightarrow \min$$

Алгоритм K-Means.

1. Инициализация: случайным образом выбираем K центров кластеров $\mu_1, \dots, \mu_K$.
2. Повторяем до сходимости:
 - (а) Каждому объекту присваиваем кластер, чей центр ближе всего:

$$C_k = \operatorname{argmin}_{k=1 \dots K} \|x_i - \mu_k\|$$

- (б) Пересчитываем центры кластеров как среднее по всем объектам кластера:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i.$$

Под сходимостью обычно понимается выполнение одного из нескольких критериев:

1. Центры кластеров перестают сильно меняться

$$\max_{k=1 \dots K} \|\mu_k^{(t+1)} - \mu_k^{(t)}\| < \delta$$

2. **Назначение точек по кластерам не изменяется**
3. **Достигнуто максимальное число итераций.** Обычно мы не хотим, чтобы алгоритм обрабатывал бесконечно и ставим в качестве параметра максимально возможное число итераций алгоритма `max_iter`

Особенности:

- Чувствителен к выбору начальных центров.
- Требуется заранее заданное число кластеров K .
- Эффективен для сферических кластеров одинаковой плотности.

Упр. 1. Часто говорят, что шаги (а) и (б) в алгоритме K-Means схожи с шагами в EM-алгоритме (Е и М, соответственно). Подумайте, какая связь между ними и чем они похожи?

§2.2 DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) — алгоритм кластеризации на основе плотности. Кластеры формируются как области высокой плотности точек, а области низкой плотности считаются шумом.

Основные понятия:

- **Ядро (core point)** — точка, у которой в ε -окрестности есть минимум `minPts` точек.
- **Граничная точка (border point)** — точка, которая находится в ε -окрестности ядра, но сама не является ядром.
- **Шумовая точка (noise point)** — точка, не являющаяся ядром и не принадлежащая к кластерам.

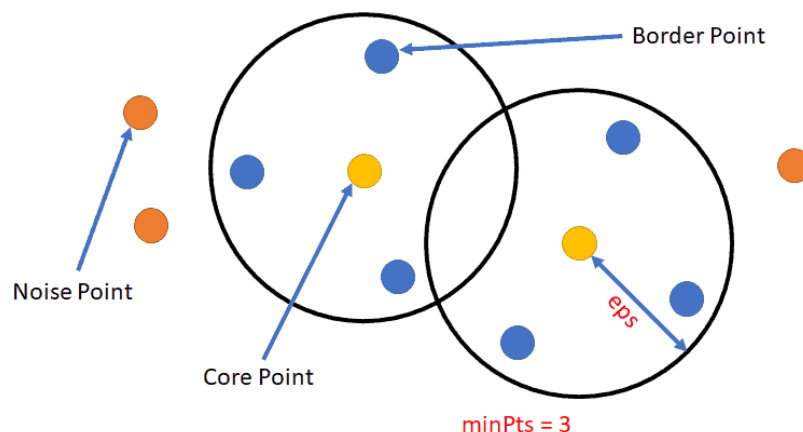


Рис. 1. Пример ядра, шумовой и граничной точки

Алгоритм DBSCAN:

1. Для каждой точки x вычисляем ε -окрестность.
2. Если x — ядро, создаём новый кластер и добавляем все точки из ε -окрестности.
3. Расширяем кластер рекурсивно, добавляя новые ядра и их ε -окрестности.
4. Повторяем для всех точек. Точки, не вошедшие ни в один кластер, считаются шумом.

Более формально:

1. Для каждой точки $x_i \in X$ определить ее ε -окрестность

$$N_\varepsilon(x_i) = \{x_j : |x_i - x_j| \leq \varepsilon\}$$

2. Пометить все точки как *не посещённые*.
3. Для каждой непосещённой точки x_i :
 - (а) Отметить x_i как посещённую.
 - (б) Если $|N_\varepsilon(x_i)| \geq \text{minPts}$, создать новый кластер C и добавить x_i .
 - (в) Рекурсивно добавить в C все точки из $N_\varepsilon(x_i)$, которые также являются ядрами.
 - (г) Если $|N_\varepsilon(x_i)| < \text{minPts}$, пометить x_i как шум.
4. Повторять, пока все точки не будут посещены.

Особенности:

- Не требует заранее задавать число кластеров.
- Может находить кластеры произвольной формы.
- Чувствителен к выбору ε и minPts .

§2.3 Иерархическая кластеризация

Идея иерархической кластеризации состоит в том, что мы хотим вместо обычного разделения на кластеры строить целую структуру в виде дерева, где можно рассматривать данные на разных уровнях детализации. Такой подход особенно полезен, когда кластеры имеют вложенную природу: крупные группы могут содержать подгруппы, а те — ещё более мелкие объединения. Например, при анализе пользователей социальной сети: мы можем выделить группы «активных» и «редко заходящих». Затем, внутри группы активных мы можем разделить пользователей по интересам («геймеры», «любители кино» и т.д.), а затем ещё глубже разделить «любителей кино» по любимым жанрам фильмов.

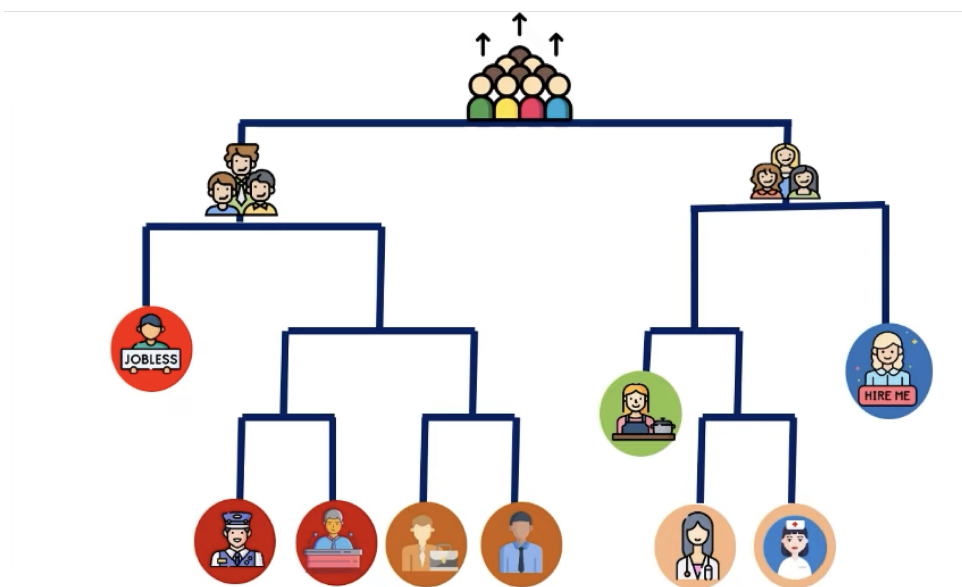


Рис. 2. Иерархическая кластеризация на примере групп людей

В иерархической кластеризации выделяют два подхода: *агломеративный* и *дивизивный*. Оба решают одну и ту же задачу, однако подходят к ней с разных сторон.

Агломеративный подход:

1. Каждая точка начинается как отдельный кластер.
2. На каждом шаге объединяются два ближайших кластера по выбранной метрике.
3. Процесс повторяется, пока все объекты не объединятся в один кластер.

Дивизивный подход:

1. Все объекты начинают в одном кластере.
2. Кластер последовательно делится на два под-кластера до достижения нужного числа.

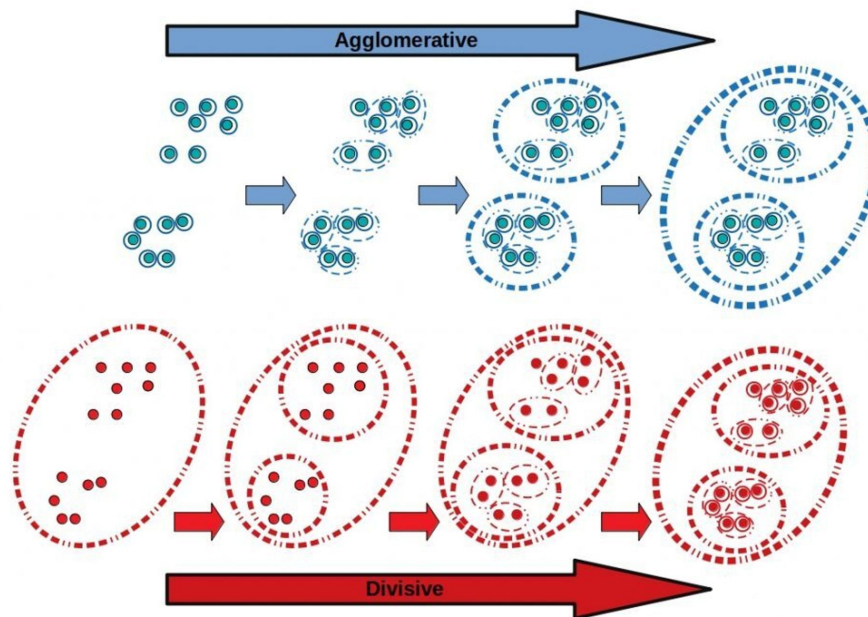


Рис. 3. Агломеративный vs дивизивных подход

Метрики расстояний между кластерами:

- **Single-link:** расстояние между ближайшими точками кластеров.
- **Complete-link:** расстояние между самыми удалёнными точками.
- **Average-link:** среднее расстояние между всеми парами точек.

§2.4 Другие подходы

- **Gaussian Mixture Models (GMM):** данные моделируются как смесь нормальных распределений; используется ЕМ-алгоритм для оценки параметров каждого компонента.
- **Spectral Clustering:** строится матрица схожести объектов, вычисляются собственные векторы, затем на них применяется K-Means.
- **Mean Shift:** ищет области максимальной плотности данных, сдвигая точки к локальным максимумам плотности.

Эти методы позволяют обрабатывать более сложные структуры данных и часто дают более гибкие результаты по сравнению с классическими алгоритмами вроде K-Means.

3 Методы оценки качества кластеризации

Оценка качества кластеризации — одна из самых сложных и важных частей анализа, поскольку в отличие от задач обучения с учителем, у нас обычно отсутствуют истинные метки классов. Поэтому важно иметь способы понять, насколько

хорошо алгоритм разделил данные на группы. Методы оценки качества можно разделить на три категории: *внутренние* (без эталона), *внешние* (с эталоном) и *визуальные* (на глаз).

§3.1 Метрики без эталона (внутренние)

Внутренние метрики оценивают качество кластеризации, исходя только из самих данных и полученных кластеров (то есть без размеченных данных). Они измеряют, насколько плотны кластеры и насколько хорошо они отделены друг от друга. Пусть $d(x_i, x_j)$ обозначает расстояние между объектами x_i и x_j .

Примеры метрик:

- Среднее внутрикластерное расстояние:

$$W = \frac{1}{K} \sum_{k=1}^K \frac{1}{|C_k|^2} \sum_{x_i, x_j \in C_k} d(x_i, x_j),$$

где C_k — множество точек кластера k . Чем меньше W , тем плотнее кластеры.

- Среднее межкластерное расстояние:

$$B = \frac{2}{K(K-1)} \sum_{i < j} d(\mu_i, \mu_j),$$

где μ_i — центр кластера i . Чем больше B , тем лучше кластеры отделены друг от друга.

- Коэффициент силуэта (*Silhouette Coefficient*):

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)},$$

где a_i — среднее расстояние от точки x_i до всех точек своего кластера, а b_i — минимальное среднее расстояние до точек ближайшего другого кластера. Среднее значение s_i по всем точкам показывает качество кластеризации: чем ближе к 1, тем лучше.

Эти метрики особенно полезны, когда истинные метки отсутствуют, и позволяют выбрать оптимальное число кластеров K или сравнить несколько алгоритмов между собой.

§3.2 Метрики с эталоном (внешние)

Метрики этого типа используются в тех случаях, когда известны «правильные» метки кластеров $Y = \{y_1, \dots, y_n\}$, то есть эталонное разбиение данных. Пусть алгоритм кластеризации выдал предсказанные метки $\hat{Y} = \{\hat{y}_1, \dots, \hat{y}_n\}$. Цель внешних метрик — оценить, насколько хорошо найденное разбиение совпадает с истинным.

Rand Index (RI). Пусть n_{11} — число пар объектов, попавших в один кластер и в эталонном, и в предсказанном разбиении, n_{00} — число пар, оказавшихся в разных кластерах в обоих разбиениях, а n_{10} и n_{01} — число несогласованных пар. Тогда **индекс Рэнда** вычисляется как

$$RI = \frac{n_{11} + n_{00}}{n_{11} + n_{00} + n_{10} + n_{01}}.$$

Значение RI находится в диапазоне $[0, 1]$, где 1 соответствует полному совпадению кластеризаций.

Adjusted Rand Index (ARI). Так как индекс Рэнда не учитывает случайные совпадения, используется его нормализованная версия — **скорректированный индекс Рэнда**:

$$ARI = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}},$$

где n_{ij} — количество объектов, принадлежащих кластеру i в эталоне и кластеру j в предсказании, $a_i = \sum_j n_{ij}$, $b_j = \sum_i n_{ij}$, а n — общее число объектов. Значение $ARI = 1$ соответствует идеальному совпадению, а $ARI \approx 0$ — результату, не лучше случайного.

Mutual Information (MI). Взаимная информация измеряет степень зависимости между двумя разбиениями:

$$MI(Y, \hat{Y}) = \sum_{i=1}^K \sum_{j=1}^L \frac{n_{ij}}{n} \log \frac{n_{ij} n}{a_i b_j}.$$

Чем выше значение MI, тем сильнее корреляция между истинными и предсказанными кластерами.

Normalized Mutual Information (NMI). Для удобства сравнения MI нормируют:

$$NMI(Y, \hat{Y}) = \frac{MI(Y, \hat{Y})}{\sqrt{H(Y) H(\hat{Y})}},$$

где $H(Y)$ и $H(\hat{Y})$ — энтропии распределений кластеров. Значение NMI находится в диапазоне $[0, 1]$.

Adjusted Mutual Information (AMI). Скорректированная взаимная информация устраняет влияние случайных совпадений:

$$AMI(Y, \hat{Y}) = \frac{MI(Y, \hat{Y}) - \mathbb{E}[MI(Y, \hat{Y})]}{\max\{H(Y), H(\hat{Y})\} - \mathbb{E}[MI(Y, \hat{Y})]}.$$

Интерпретация. Метрики с эталоном позволяют количественно оценивать, насколько хорошо алгоритм воспроизводит известное истинное разбиение. ARI и AMI считаются наиболее устойчивыми и часто используются в экспериментах, так как учитывают случайные совпадения и не зависят от числа кластеров.

§3.3 Метод локтя

Одной из ключевых задач при использовании алгоритмов кластеризации, таких как K-Means, является выбор оптимального числа кластеров K . Если задать слишком маленькое значение K , то в одном кластере окажутся объекты, существенно отличающиеся друг от друга. Если же выбрать слишком большое K , то кластеры будут избыточно дробными, и модель начнёт «подгоняться» под шум.

Идея метода. Метод локтя (*Elbow Method*) основан на анализе зависимости значения целевой функции кластеризации от числа кластеров K . Для каждого значения K вычисляется суммарное внутрикластерное расстояние:

$$J(K) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2,$$

где μ_k — центр k -го кластера.

Затем строится график зависимости $J(K)$ от числа кластеров K . По мере увеличения K значение $J(K)$ монотонно убывает, так как кластеры становятся всё более «плотными». Однако начиная с некоторого момента добавление новых кластеров приводит лишь к незначительному уменьшению $J(K)$. В этот момент, как правило, начинается излишнее «дробление» кластеров. Точка, где скорость убывания резко меняется (образуя характерный «излом» или «локоть»), и принимается за оптимальное значение K (см. Рис. 4).

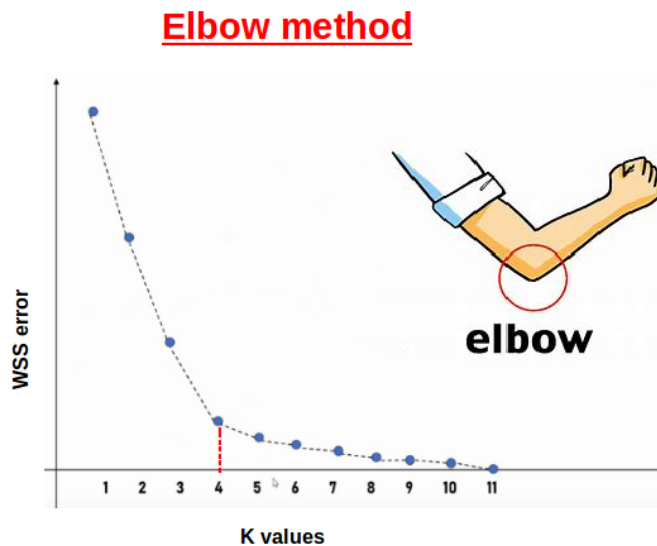


Рис. 4. Иллюстрация метода локтя для выбора оптимального числа кластеров.

Интерпретация. Графически метод локтя позволяет визуально определить баланс между качеством кластеризации и её сложностью. Значение K , соответствующее «локтю», обеспечивает хорошее приближение данных без излишнего дробления на кластеры.

4 Визуализация и интерпретация кластеров

Когда данных немного или пространство признаков можно визуализировать (например, после снижения размерности с помощью **t-SNE** или **UMAP**), полезно использовать визуальные методы оценки кластеризации. Они позволяют интуитивно оценить:

- плотность и форму кластеров;
- степень пересечения между ними;
- наличие шумовых или выбросных точек.

Типичный приём — построение двумерного отображения данных с цветовой разметкой кластеров. Если кластеры визуально отделяются и имеют компактную форму, это указывает на хорошее качество кластеризации. Однако визуальная оценка субъективна и должна рассматриваться лишь как вспомогательный инструмент.

§4.1 t-SNE

Метод **t-distributed Stochastic Neighbor Embedding (t-SNE)** [1] предназначен для нелинейного снижения размерности и визуализации высокоразмерных данных. Основная идея заключается в том, чтобы сохранить *локальную структуру* данных: точки, близкие в исходном пространстве, должны оставаться близкими и после проекции.

Метод моделирует вероятности близости объектов как в исходном, так и в целевом (обычно двумерном) пространстве, и минимизирует расхождение между ними с помощью дивергенции Кульбака–Лейблера:

$$\text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}},$$

где p_{ij} и q_{ij} — вероятности того, что точки x_i и x_j являются соседями в исходном и низкоразмерном пространстве соответственно.

t-SNE хорошо выделяет локальные кластеры, но не сохраняет глобальную структуру данных (например, расстояния между кластерами). Результаты чувствительны к параметрам и случайной инициализации.

§4.2 UMAP

Uniform Manifold Approximation and Projection (UMAP) [2] — современный метод снижения размерности, основанный на идеях топологического анализа данных. UMAP, как и t-SNE, стремится сохранить локальную структуру, но делает

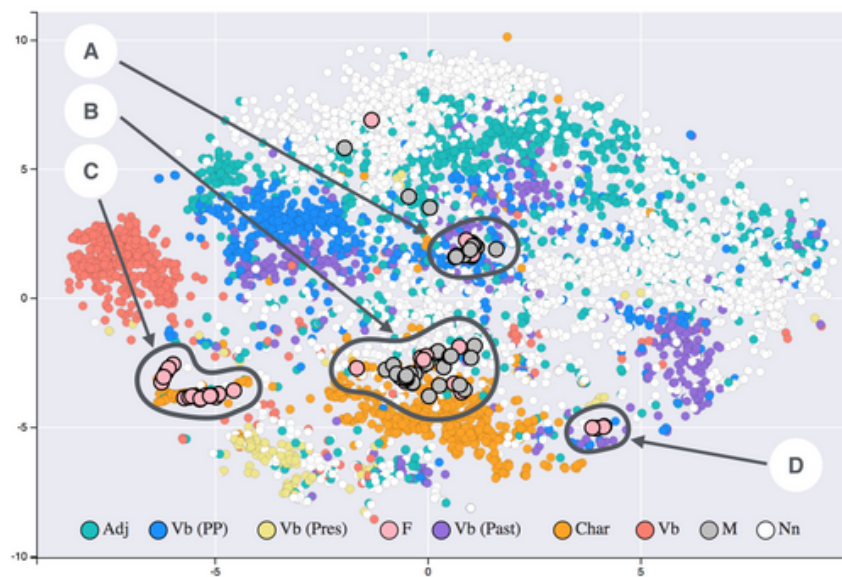


Рис. 5. t-SNE визуализация эмбедингов слов, обученных на литературе 19 века с

это более эффективно и с возможностью сохранить глобальные отношения между кластерами.

Метод строит приближение *многообразия* данных в виде графа соседей, а затем оптимизирует низкоразмерное представление так, чтобы сохранить топологию этого графа. UMAP работает быстрее t-SNE, масштабируется на большие выборки и лучше отражает взаимное расположение кластеров.

На практике UMAP часто используется для предварительной визуализации результатов кластеризации или для уменьшения размерности перед применением алгоритмов вроде DBSCAN.

§4.3 PCA

Principal Component Analysis (PCA) — один из классических методов линейного снижения размерности. Он находит такие направления (главные компоненты), вдоль которых проекция данных имеет наибольший разброс. Формально задача PCA сводится к поиску ортогонального преобразования, которое максимизирует дисперсию проекции:

$$\max_W \text{Tr}(W^T S W), \quad \text{при } W^T W = I,$$

где S — ковариационная матрица данных.

PCA сохраняет глобальную структуру данных и может быть полезен для первичного анализа, но плохо отображает нелинейные зависимости. Тем не менее, визуализация кластеров в пространстве первых двух компонент часто даёт простое и наглядное представление о разделимости классов.

На рисунке 6 визуально показано, как выглядит визуализация кластеров с помощью t-SNE, UMAP и PCA. Более подробнее почитать про различия между PCA, UMAP и t-SNE можно в этой статье на [Хабре](#).

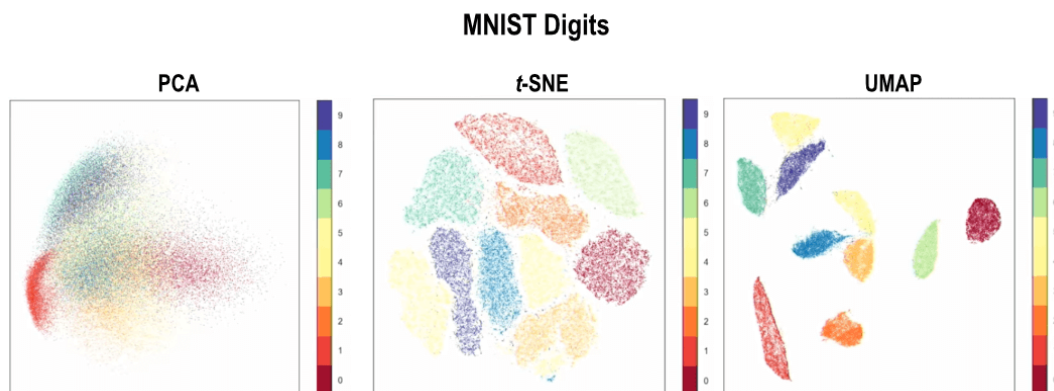


Рис. 6. Визуальное различие между t-SNE, UMAP и PCA на датасете MNIST

5 Практические применения

§5.1 Кластеризация текстов

Одним из наглядных примеров практического применения кластеризации является **кластеризация текстов**. Она широко используется в задачах обработки естественного языка (NLP) — например, при анализе пользовательских запросов, группировке новостей, тематическом моделировании или в системах поддержки клиентов.

Рассмотрим прикладной пример. Предположим, мы разрабатываем чат-бота (ИИ-ассистента) для банковского приложения. Такой бот должен уметь распознавать тип запроса пользователя и подбирать соответствующий сценарий ответа. Например:

- «Как мне оформить кредит на трактор?» — класс *оформление кредита*;
- «Почему мне не пришёл кешбэк за покупку?» — класс *проблемы с кешбэком*;
- «Как сменить номер телефона в приложении?» — класс *изменение личных данных*.

Чтобы построить классификатор запросов, нужно заранее определить возможные категории (классы). Однако на практике заранее неизвестно, какие типы вопросов встречаются чаще всего, и как именно они формулируются пользователями. Поэтому перед этапом классификации полезно провести **кластеризацию текстов** — чтобы автоматически выделить группы похожих запросов и определить наиболее распространённые категории обращений.

Таким образом, кластеризация позволяет:

- обнаружить основные тематики в базе пользовательских запросов;
- уменьшить дублирование и разнообразие формулировок;
- облегчить последующую разметку данных для обучения модели классификации.

Представление текстов в векторном виде. Пусть у нас есть множество текстов

$$T = \{t_1, t_2, \dots, t_n\}.$$

Для применения алгоритмов кластеризации каждый текст необходимо преобразовать в числовое векторное представление:

$$e : t_i \mapsto v_i \in \mathbb{R}^d.$$

Существует несколько распространённых способов построения таких представлений.

- **Bag-of-Words (BoW).** Каждый текст описывается бинарным вектором размерности $|V|$, где V — словарь всех уникальных слов. На позиции j ставится 1, если слово j встречается в тексте, и 0 — если нет. Метод прост, но не учитывает порядок слов и частоты их употребления, а также плохо масштабируется при большом словаре.
- **TF-IDF (Term Frequency–Inverse Document Frequency).** Более информативный способ, при котором учитывается не только наличие слова, но и его важность в корпусе текстов. Для слова w в тексте t значение TF-IDF вычисляется как:

$$\text{tf-idf}(w, t) = \text{tf}(w, t) \cdot \log \frac{N}{\text{df}(w)},$$

где $\text{tf}(w, t)$ — частота слова w в тексте t , $\text{df}(w)$ — количество документов, содержащих w , а N — общее число текстов. TF-IDF векторизация часто используется как базовый подход в сочетании с алгоритмом K-Means.

- **Нейросетевые эмбединги.** Современные модели позволяют получать плотные векторные представления текстов, в которых семантически близкие тексты оказываются близкими и в векторном пространстве. Классические подходы включают модели Word2Vec и GloVe, которые обучают эмбединги слов, из которых затем можно получить представление текста как среднего по словам. Более продвинутые модели, такие как **BERT**, **Sentence-BERT (SBERT)** или **Universal Sentence Encoder (USE)**, позволяют напрямую получать эмбединги предложений и документов, отражающие их смысл. Такие вектора можно эффективно кластеризовать алгоритмами K-Means, DBSCAN или HDBSCAN.

Как находить категории. Пусть у нас есть корпус из тысяч пользовательских запросов. После преобразования текстов в эмбединги (например, с помощью Sentence-BERT) применим алгоритм K-Means и получим, скажем, $K = 30$ кластеров. Просмотрев примеры из каждого кластера (можно, например, воспользоваться методами визуализации наиболее популярных слов, таких как wordcloud), мы можем вручную присвоить им осмысленные метки: *платёжные операции*, *технические проблемы*, *вопросы о картах*, *сброс пароля* и т.д. В дальнейшем эти кластеры можно использовать для обучения модели классификации запросов.

Еще один пример применения кластеризации текстов Автор данного текста, кем бы он ни был, однажды столкнулся на работе с практической задачей: необходимо было обучить модель типа *cross-encoder*, которая для пары (пользовательский запрос, описание товара) выдавала бы оценку релевантности на шкале от -1 до 1. Целью было улучшить качество работы маркетплейса.

Возникла следующая проблема: маркетплейс был молодым, и база пользовательских запросов была относительно небольшой, что делало обучение кросс-энкодера неэффективным. Для решения этой задачи было решено использовать более крупный набор запросов другого популярного сервиса, который согласился предоставить данные. Однако распределение тем в этом наборе существенно отличалось от распределения запросов на нашем маркетплейсе, и модель по-прежнему обучалась плохо.

Для исправления ситуации было предложено следующее решение: с помощью кластеризации выделить около 20 наиболее популярных категорий запросов (например, мебель, товары для кухни, еда, электроника, мода и др.) и скорректировать распределение большого внешнего датасета таким образом, чтобы оно приближалось к распределению запросов на нашем маркетплейсе. После такой предобработки модель-кросс-энкодер обучалась значительно эффективнее и демонстрировала более высокое качество предсказаний.

§5.2 Кластеризация гео-данных

Кластеризация гео-данных используется для выявления пространственных закономерностей и группирования объектов, близких по расположению. Такие задачи часто встречаются в городском планировании, анализе транспортных потоков, маркетинге и экологии.

Постановка задачи. Пусть у нас есть набор точек на карте

$$X = \{(lat_i, lon_i)\}_{i=1}^n,$$

где каждая точка соответствует, например, месту проживания пользователя, вызову такси или зарегистрированному случаю события (продажа, ДТП, выброс и т.д.). Наша цель — выделить регионы (кластеры), в которых наблюдается высокая концентрация точек.

Метод. Для таких данных часто используется алгоритм **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise), так как он хорошо работает с пространственными координатами и способен обнаруживать кластеры произвольной формы. Кроме того, он автоматически определяет количество кластеров и устойчив к шуму, что особенно важно для географических данных.

Пример. На практике кластеризация гео-данных может применяться для:

- выделения популярных зон вызова такси;
- определения районов с повышенной активностью пользователей мобильных приложений;

- анализа распределения экологических измерений (например, зон с повышенным загрязнением воздуха);
- обнаружения пространственных аномалий.

В [Яндекс Хендбуке](#) можно более подробно почитать, как кластеризация гео-данных использовалась при разработке пикап-пойнтов (наиболее удобных точек вызова такси, подсвечиваемых в приложении Яндекс.Такси).

Список литературы

- [1] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.
- [2] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.